SAEMPHYSICS

AI Animation Studio

Architecture, Engineering & Production Workflow Textbook

A practical build manual for visual physics education
NEET • JEE Main/Advanced • Physics Olympiads

Inspired by conceptual teaching, mathematical animation, and deep explanatory lectures.
Built around Python, Manim, GitHub, Codespaces, Colab, and AI assistants.

Executive blueprint

This textbook describes a maintainable production system—not a one-click AI video generator. The teacher owns the physics and pedagogy; AI assists; code makes visual behavior repeatable; Git preserves the work; and explicit quality gates protect the final lesson.

- First milestone: render one correct, understandable 30–60 second scene.
- Second milestone: create reusable components for equations, vectors, axes and graphs.
- Third milestone: publish one complete short episode using the full pipeline.
- Fourth milestone: make the process reproducible and improve it from measured bottlenecks.

The production pipeline at a glance

Stage	Input	Output / approval
Editorial	Topic and audience	Brief and learning objective
Physics	Question and model	Verified derivation and assumptions
Design	Approved physics	Script, storyboard and scene cards
Animation	Scene cards	Manim code and preview renders
Post-production	Approved visuals + voice	Edited video, captions and mix
Release	Final export	QC-approved video and archived source

How to use this textbook

Purpose

Use this manual as both a learning sequence and a build specification. Read the architecture first, then implement one small vertical slice: a correct physics idea, one animation, one narration track, and one exported video. Do not begin by automating everything.

The guiding rule

Physics correctness comes before visual polish; clarity comes before complexity; repeatability comes before speed. AI can accelerate drafting and coding, but the teacher remains the subject-matter authority and final editor.

Definition of done

A production is complete when the physics has been checked, the intended learner outcome is clear, the scene renders reliably, narration is intelligible and synchronized, the final file plays correctly, and the project can be reproduced from its repository.

How to work through it

At the end of each chapter, complete the deliverable and checklist. Keep a decision log. When a step fails, record the error and the fix rather than relying on chat history alone.

1. Vision, principles and scope

1.1 The product

Build a repeatable studio for producing visual physics explanations with the conceptual clarity of Feynman-style teaching, the mathematical animation sensibility associated with 3Blue1Brown, and the deeper theoretical connections found in advanced lecture courses. Take inspiration from methods, not another creator's exact artwork, voice, scripts, or branding.

1.2 Target audiences

Maintain three audience modes. NEET: intuition, standard models, units, graphs, and fast but justified problem-solving. JEE Main/Advanced: derivations, assumptions, limiting cases, multi-step reasoning, and traps. Olympiads: modeling, proof, estimation, alternate approaches, and extensions beyond routine syllabus patterns.

1.3 Editorial principles

Every video should answer one central question. Introduce a puzzle before the formalism. Explain what symbols mean. Make assumptions visible. Use animation to reveal relationships that are hard to see in a static diagram. Return to the opening question at the end. Include practice only after the core idea is established.

1.4 Non-goals for the first month

Do not build an autonomous video factory, a custom web application, a complex 3D renderer, or a multi-agent orchestration system. Do not pay for multiple AI subscriptions before identifying a real bottleneck. Do not generate an entire episode from one giant prompt.

Deliverable

Write a one-page channel brief: audience, language, episode length, visual identity, learning outcomes, and what makes a SaemPhysics episode distinct.

2. Reference architecture

2.1 System overview

The system is a pipeline of small, testable stages. Each stage has explicit inputs and outputs. Keep source files and decisions in Git; keep large rendered outputs and raw recordings in an organized asset store, not indiscriminately in source control.

Architecture layers

A. Editorial layer: topic, learner, misconception, objective, narrative. B. Physics layer: definitions, assumptions, derivations, equations, numerical checks. C. Scene specification: ordered beats, visuals, equations, timing, narration cues. D. Animation layer: reusable Manim components and scene code. E. Render layer: low-resolution previews and final video. F. Audio/editing layer: narration, cleanup, mix, captions, final assembly. G. Quality and release layer: verification, accessibility, export, metadata, archive.

2.2 Recommended technology stack

Core: Python, Manim Community Edition, NumPy, SymPy, Matplotlib when useful, FFmpeg, Git and GitHub. Cloud prototyping: Google Colab. Cloud development: GitHub Codespaces. Editing: DaVinci Resolve or another editor you already know. Audio: a microphone and Audacity or equivalent. AI assistants: Gemini and DeepSeek as interchangeable collaborators, not authorities.

2.3 Tool responsibility matrix

Gemini: brainstorm explanations, compare analogies, outline scripts, critique clarity. DeepSeek: propose derivations, write or debug code, suggest tests. Python/SymPy: calculate and symbolically inspect results. NumPy: numerical simulation. Manim: deterministic mathematical animation. Colab: fast experiments and notebooks. Codespaces/local environment: maintainable project development. GitHub: version history, issues, reviews and release tags. Video editor: pacing, narration, captions, sound and final assembly. Teacher: scientific approval and pedagogical judgment.

2.4 Data flow

Topic brief → physics note → script → storyboard → scene specification → code → preview render → technical and scientific review → narration/edit → final QC → publication package. A correction upstream must trigger a review of downstream artifacts. For example, changing a sign convention can affect the derivation, on-screen equation, code, narration, and quiz.

2.5 Human approval gates

Gate 1: approve the learning objective and central question. Gate 2: approve physics and derivation. Gate 3: approve storyboard and visual encoding. Gate 4: approve preview animation. Gate 5: approve final export. AI must not silently bypass these gates.

Deliverable

Draw this pipeline in a README and identify who or what approves each stage. Initially, that person is you.

3. Set up your working environment

3.1 Choose a primary environment

For a beginner, use a local computer if it can run Python and render video; otherwise prototype in Colab and use Codespaces for repository-based development. Cloud sessions can reset, storage can be temporary, and rendering performance varies. Keep a tested local or cloud fallback.

3.2 Installation strategy

Install Python using a current supported version compatible with your selected Manim release. Create a virtual environment per project. Install dependencies from a requirements file. Follow the official Manim Community installation guide for platform-specific system dependencies rather than copying unverified commands from an AI response.

3.3 First environment checks

Run `python --version`, `git --version`, and `manim --version`. Run the official sample scene before building your own. Check that FFmpeg and any LaTeX dependencies required by your chosen text workflow are available. Save the versions and successful setup steps in README.md.

3.4 Colab versus Codespaces

Colab is best for a quick numerical experiment, plotting data, or sharing a notebook. Codespaces is better for a structured multi-file repository, repeatable commands, code review, and development close to GitHub. Neither guarantees unlimited compute or permanent storage. Test a small render before committing to long scenes.

3.5 AI-generated code safety

Never paste secrets, API keys, private student data, or credentials into prompts or repositories. Read commands before executing them. Avoid running shell commands with destructive flags unless you understand the impact. Check package names and install instructions against official documentation.

Deliverable

A clean environment that renders a sample scene from a fresh checkout, with setup instructions written down.

4. Repository and asset architecture

4.1 Suggested project tree

Use a predictable structure so that future videos do not become piles of unrelated files.

Directory tree

```
saemphysics-studio/
  README.md
  pyproject.toml or requirements.txt
  .gitignore
  config/
    brand.yaml
    render.yaml
  docs/
    architecture.md
    decisions/
  templates/
    episode_brief.md
    physics_review.md
    storyboard.md
    release_checklist.md
  library/
    components/
    physics/
    styles/
  episodes/
    electrostatics_001/
      brief.md
      physics_notes.md
      script.md
      storyboard.md
      scenes.py
      tests/
      assets/
        audio/
        images/
      exports/          # usually ignored by Git
      README.md
  notebooks/
  scripts/
```

4.2 What belongs in Git

Commit source code, scripts, small configuration files, documentation, prompts, tests, and small reusable assets with clear licensing. Usually exclude caches, temporary render files, huge video exports, private raw audio, and generated files that can be recreated. For large assets, use a deliberate storage policy or Git LFS if appropriate.

4.3 Naming conventions

Use stable, descriptive identifiers such as `electrostatics_001`, `scene_03_superposition`, and `v001`. Avoid names like `final_final_new2`. Use release tags for approved versions. Keep a short changelog for meaningful changes.

4.4 Reproducibility

Record Python and library versions, render command, resolution, frame rate, random seeds when randomness is used, and any external assets. Keep the episode's inputs and approval status clear. A future teacher should be able to identify exactly which source generated a published video.

Deliverable

Create the repository, README, .gitignore, episode template, and first Git commit before writing a full episode.

5. The physics verification layer

5.1 Start from a claim, not an equation

Write the physical question in plain language. Define the system, the observer or reference frame, the quantities involved, and the assumptions. Only then choose the governing model.

5.2 A standard verification protocol

For each central result: state definitions; list assumptions; derive the equation step by step; check dimensions; inspect signs and directions; test limiting cases; compare against an independent derivation or numerical example; identify the domain where the formula applies; and ask what misconception a student might take away.

5.3 Use AI as a challenger

Ask one assistant to propose a derivation and another to critique it. Then verify independently. A second model agreeing with the first is not proof: both can reproduce the same error. For high-stakes exam content, compare with trusted textbooks, official syllabus materials, or your own checked solution.

5.4 Symbolic and numerical checks

SymPy can simplify expressions and differentiate or integrate symbolic forms. NumPy can test many values and plot behavior. Neither tool understands every physical assumption automatically. A dimensionally consistent expression can still be physically wrong; numerical agreement at a few points can conceal an incorrect model.

5.5 Example: point-charge field

For a point charge Q in vacuum, at a position vector r measured from the charge, the electric field is $E(r) = (1/(4\pi \varepsilon_0)) Q \hat{r}/r^2$. State that the expression applies away from the source point and that the direction reverses for negative Q. Check units: the result has units N/C. As r doubles, magnitude falls by a factor of four. The vector form and the magnitude form should not be mixed without explanation.

5.6 Physics review record

For each episode, record: claim, equation, assumptions, source or derivation, dimensional check, limiting-case check, numerical test, likely student misconception, reviewer, date, and status. Mark unresolved uncertainty openly instead of smoothing it over in narration.

Deliverable

A completed physics review sheet for one result that will appear on screen.

6. Pedagogy and narrative design

6.1 The conceptual arc

Use this flexible arc: hook → intuition → representation → formal model → derivation → test or counterexample → exam connection → recap. Not every video needs every stage, but every stage included must earn its time.

6.2 Feynman-inspired explanation

Explain an idea in ordinary language, identify what is actually difficult, use a concrete example, and expose assumptions. Avoid pretending an analogy is the mechanism itself. State where the analogy breaks down.

6.3 Mathematical animation principles

A moving object should encode a meaningful variable. A vector's length should have a defined relation to magnitude. Axes, units and reference frames should be clear. Use consistent visual grammar: for example, one stable color per physical quantity. Do not rely on color alone; use labels, shapes or line styles as well.

6.4 Script format

Write narration and visuals side by side. Each beat should contain: time estimate, spoken line, visual action, equation or label, intended inference, and transition. The viewer should not be asked to read a dense equation while the narrator introduces a different idea.

6.5 Audience adaptation

The same animation can support different levels. For NEET, emphasize interpretation and standard use. For JEE Advanced, reveal the assumptions and reasoning steps. For Olympiad, add an extension, alternative proof, or open-ended prediction. Do not turn every episode into three full lectures; use chapters or companion videos.

Deliverable

A script in which every visual has a teaching purpose and every equation is explained in words.

7. Storyboarding and scene specification

7.1 Storyboard before code

A storyboard prevents code from becoming the place where you discover the lesson. Sketch the screen, identify the object that moves, determine what the learner should notice, and decide how the scene transitions.

7.2 Scene specification schema

For each scene, record: ID, objective, duration target, narration cue, starting state, visual elements, animation actions, equations, labels and units, camera/frame assumptions, transition, correctness risks, and completion test.

7.3 One scene, one job

Prefer small scenes with one conceptual purpose. A scene might introduce a charge, show its field vector, then compare two distances. Keep reusable mathematical objects in library components. Avoid a single giant scene with tangled logic and hard-coded timings.

7.4 Visual review questions

Can a student identify the object, axes, direction and units? Is the encoding consistent? Does the animation imply causation that the model does not support? Does the motion clarify a relationship or merely decorate it? Can the scene be understood without sound, at least at a basic level?

Deliverable

A storyboard with scene IDs that map directly to code and review notes.

8. Building with Manim

8.1 The basic idea

Manim describes objects and transformations in Python and renders them into frames. You write a scene, render a preview, inspect it, adjust timings and layout, then render final quality. Learn the core abstractions before trying advanced camera moves or complex 3D scenes.

8.2 Suggested learning sequence

First: text, equations, lines, dots and arrows. Second: create, transform, move and fade animations. Third: coordinate planes, graphs and trackers. Fourth: vectors, parameterized curves and numerical data. Fifth: reusable components, scene composition, custom styles and tests. Sixth: optimization and production rendering.

8.3 Reusable components

Build small helpers for labeled vectors, axes with units, equation reveals, charge markers, graph traces, callout boxes, and consistent title cards. Helpers should have documented inputs and predictable behavior. Do not abstract a component until you have used it enough to understand what should be configurable.

8.4 Coordinate systems and units

Distinguish screen coordinates from physical coordinates. A diagram may be schematic, but if distances or slopes are quantitatively meaningful, encode the scale correctly or clearly label the visualization as schematic. Keep a single source of truth for physical parameters used by equations and animations.

8.5 Animation timing

Render low-quality previews early. Use named constants for timing and visual parameters where helpful. Avoid unnecessary redraws or recomputing expensive quantities each frame. For simulations, calculate state carefully and separate the physical model from the visual representation.

8.6 A tiny starter scene

The following is a minimal pattern to adapt after installing Manim. Test it with the render command shown in your project README.

Code example

```
from manim import *

class FirstPhysicsScene(Scene):
    def construct(self):
        title = Text("Electric field", font_size=42)
        charge = Dot(ORIGIN, color=YELLOW)
        label = MathTex("+Q").next_to(charge, DOWN)
        field = Arrow(
            start=ORIGIN,
            end=2 * RIGHT,
            buff=0,
            color=BLUE,
            stroke_width=6,
        )
        field_label = MathTex(r"\vec E").next_to(field, UP)
        self.play(Write(title.to_edge(UP)))
        self.play(FadeIn(charge), Write(label))
        self.play(GrowArrow(field), Write(field_label))
        self.wait(1)
```

8.7 Render and inspect

Use the command syntax documented by your installed Manim version, for example `manim -pql scenes.py FirstPhysicsScene` for a low-quality preview in a typical Manim Community setup. Confirm the output location and options against the installed version. Watch the render at normal speed, not just as a still image.

Deliverable

A short scene with one equation, one physical object, one vector, labels, and a clean render.

9. Modeling and simulation

9.1 Separate model from drawing

Create a function or module that calculates physical state, then let the animation display that state. This separation makes it easier to test the physics without rendering video and easier to change the visual design without rewriting the model.

9.2 Use appropriate tools

Use SymPy for symbolic derivations and transformations; NumPy for arrays and numerical integration; Matplotlib for quick diagnostic plots; and Manim for animated presentation. Use a dedicated numerical solver only when the problem warrants it. Do not write a complex solver if a closed-form solution or simple time step is sufficient.

9.3 Validation tests

Compare a simulation with a known solution when available. Check conserved quantities where the model predicts conservation. Test step-size sensitivity for numerical integration. Check boundary conditions, initial conditions, units, and expected qualitative behavior.

9.4 Avoid false precision

A simulation is only as accurate as its assumptions and numerical method. State whether a display is qualitative, approximate, or quantitatively scaled. Do not imply that a numerical trajectory is an exact law.

Deliverable

A tested model module and a separate scene that visualizes its output.

10. Audio, editing and final assembly

10.1 Record narration for meaning

Write and rehearse the explanation, then record in quiet conditions with consistent microphone distance. Leave pauses for important visual changes. Record in sections so that one correction does not force a full re-record.

10.2 Audio processing

Use gentle noise reduction only when needed; excessive processing creates artifacts. Control peaks, maintain intelligibility, and keep background music below speech. Review with headphones and ordinary speakers.

10.3 Synchronization

Use scene IDs and narration cues to align edits. Leave breathing room around equations and important transitions. If the narration changes substantially, recheck timing and captions rather than stretching visuals blindly.

10.4 Captions and accessibility

Generate captions, then manually correct physics vocabulary, symbols, names, and units. Ensure equations are also explained verbally or displayed legibly. Avoid using color as the only distinction. Test mobile readability and safe margins.

10.5 Export policy

Choose a consistent aspect ratio, frame rate and resolution for the channel. Use 16:9 for standard long-form videos unless there is a deliberate reason otherwise; create a separate 9:16 composition for Shorts rather than simply cropping critical content. Verify the exported file from beginning to end.

Deliverable

One assembled episode with checked narration, captions, levels, titles and export settings.

11. AI collaboration and prompt engineering

11.1 Small tasks beat giant prompts

Ask an AI assistant to produce one well-defined artifact at a time: critique a derivation, write a storyboard, implement one scene, or propose tests. Give it the relevant source files, constraints and acceptance criteria. Ask for assumptions and uncertainties explicitly.

11.2 Physics critic prompt

You are a skeptical physics reviewer for a NEET/JEE/Olympiad lesson. Check the supplied claim and derivation. List assumptions, verify dimensions and signs, test limiting cases, identify domain restrictions and likely misconceptions. Do not silently repair gaps; show each issue and propose a corrected derivation. Separate verified facts from suggestions.

11.3 Script prompt

Create a storyboard for the stated learner and objective. For each beat give time estimate, narration, visual action, equation/labels, intended learner inference and transition. Keep one central idea per scene. Mark every scientific claim that needs teacher verification. Do not imitate any creator's exact script or signature assets.

11.4 Manim coding prompt

Implement only the requested scene using the supplied project conventions and installed Manim version. Reuse existing components where appropriate. Return complete code, imports, render command, assumptions, likely version issues, and a short test plan. Do not invent APIs; if uncertain, say so. Preserve the physics parameters and units exactly as specified.

11.5 Code review prompt

Review this diff for scientific correctness, coordinate consistency, scene layout, timing, performance, maintainability and reproducibility. Identify concrete failure modes. Suggest the smallest safe change. Do not rewrite unrelated code.

11.6 Multi-model workflow

Use one model to draft, another to critique, and deterministic code or trusted references to verify. Keep the final decision and rationale in the repository. Do not assume that two models agreeing makes a result true.

Deliverable

A versioned prompt library with a named prompt for each pipeline stage.

12. Quality assurance and testing

12.1 Four types of QA

Scientific QA checks physics and math. Pedagogical QA checks whether the learner can follow the reasoning. Visual QA checks legibility, consistency, timing and rendering artifacts. Technical QA checks export integrity, audio, captions, file naming and reproducibility.

12.2 Minimum release checklist

[ ] Learning objective is explicit. [ ] Equations and units are verified. [ ] Assumptions and limits are stated. [ ] Labels and arrows encode the intended quantities. [ ] No text is clipped or too small on mobile. [ ] Narration matches the scene. [ ] Captions are corrected. [ ] Audio is intelligible. [ ] Export plays throughout. [ ] Source and dependencies are archived. [ ] Thumbnail and metadata are reviewed.

12.3 Regression testing

When a reusable component changes, render its example scenes and inspect them. Keep small test scenes for vectors, equations, graphs and title layouts. Automated checks can catch missing files, malformed configuration or failed calculations, but visual review is still necessary.

12.4 Failure log

For each meaningful defect record: symptom, cause, fix, prevention, and affected episodes. Over time, this becomes your internal production handbook and reduces repeated debugging.

Deliverable

A release checklist and a small suite of repeatable scene tests.

13. GitHub workflow and project management

13.1 Repository habits

Commit small, coherent changes with descriptive messages. Use branches for risky experiments or major episodes. Open issues for defects and improvements. Review diffs before merging. Tag a version when an episode is approved.

13.2 Issue template

Title: [EPISODE-ID] concise task. Include goal, context, acceptance criteria, relevant files, physics risks, visual risks, and definition of done. Split large tasks into issues that can be completed and checked independently.

13.3 Continuous checks

Start with simple checks: dependency installation, Python syntax/compile checks, unit tests for model functions, and a small low-quality render. Automate more only after the manual process is stable. A green test is useful evidence, not a guarantee that an animation teaches well.

13.4 Documentation as memory

Record setup steps, design decisions, known issues and why a component exists. Chat histories are not the source of truth; the repository is. When AI proposes a change, require it to identify files changed and commands needed to verify them.

Deliverable

A repository that another technically capable person could clone and use from its README.

14. A worked production blueprint: electric fields

14.1 Brief

Working title: Electric Fields—What Changes in Space Around a Charge? Audience: JEE/NEET with an optional Olympiad extension. Core question: how can we describe the influence of a charge at locations where another charge might be placed? Learning outcomes: define electric field, interpret its vector direction, use inverse-square scaling, and apply superposition.

14.2 Physics note

Define $E = F/q_0$ for a positive test charge in the limit that the test charge does not significantly disturb the source arrangement. For a point charge, $E = kQ\,\hat{r}/r^2$. Explain the role of the sign of Q and the direction of $\hat{r}$. State the assumptions and avoid treating field lines as physical strings.

14.3 Scene plan

Scene 1: a source charge appears; ask what a second charge would experience. Scene 2: show a test charge and force direction. Scene 3: introduce E as force per unit positive test charge. Scene 4: move the test point to show inverse-square scaling. Scene 5: place two source charges and display vector superposition. Scene 6: compare vectors with field-line representation. Scene 7: solve a short exam question. Scene 8: summarize and pose a prediction question.

14.4 Implementation plan

First write and test the field calculation independently. Then create a static diagram, followed by one moving test point, then two-charge superposition. Add equation reveals only after the geometry works. Render each scene separately at low quality. Record narration after the timing is reasonably stable.

14.5 Acceptance tests

The vector direction must reverse for negative source charge. Doubling distance from a point charge must quarter field magnitude. Superposition must be vector addition, not addition of magnitudes in general. Labels must distinguish source charge, test charge, force and field. The final explanation must not imply that field lines are trajectories of all possible particles.

14.6 Reuse

Extract reusable charge markers, vector arrows, field sampling and equation-reveal helpers. The same components can support electric dipoles, equilibrium points, flux and potential, with fresh physics review for each new result.

15. Scaling the studio

15.1 Scale quality before quantity

Do not target daily polished animations before the system is stable. Begin with one short scene, then a short video, then a full episode. Track the actual time spent on scripting, verification, coding, rendering, recording and editing.

15.2 Reuse assets responsibly

Build a library of components, but review their physical meaning every time they are reused. A generic arrow component can serve different quantities only if labels, units and scale are updated. Keep reusable code separate from episode-specific content.

15.3 Episode tiers

Tier 1: simple graph or vector explainer. Tier 2: multi-scene conceptual episode. Tier 3: numerical simulation or advanced derivation. Tier 4: longer flagship documentary-style lesson. Estimate effort based on complexity and reserve review time for every tier.

15.4 Automation boundary

Automate repetitive tasks such as project scaffolding, render commands, file checks and caption drafts. Keep human approval for scientific claims, pedagogical framing, visual interpretation, final sound and publication. Avoid automatic publishing until the quality gates have been consistently reliable.

15.5 Metrics

Track production hours per finished minute, render failure rate, corrections found after preview, caption error rate, audience retention, click-through rate and student feedback. Use metrics to locate bottlenecks, not to replace educational judgment.

Deliverable

A monthly retrospective identifying one process improvement supported by evidence.

16. 90-day implementation roadmap

Days 1–7: foundation

Install or configure the environment. Create the repository. Render the first sample. Write the channel brief, project tree and episode template. Success condition: a fresh checkout can render a basic scene.

Days 8–21: core visual grammar

Build and document components for equations, labels, vectors, axes and graphs. Make five short scenes. Create physics verification and visual QA checklists. Success condition: you can assemble a short explainer without reinventing every visual.

Days 22–35: first episode

Choose one narrow concept. Write the brief, derivation, script and storyboard. Render previews, review them, record narration, edit and publish. Success condition: one finished episode and a documented retrospective.

Days 36–55: reliability

Add tests for model functions, standard render presets, a failure log, and a better project template. Improve the audio and caption workflow. Success condition: the next episode requires fewer avoidable fixes.

Days 56–75: content reuse

Create a companion short and a problem-solving segment from approved assets. Build a second episode using the same components. Success condition: reuse saves time without carrying over incorrect assumptions.

Days 76–90: controlled automation

Automate project setup, dependency checks, preview rendering and release packaging. Evaluate where AI genuinely saves time. Success condition: the process is documented, repeatable and still has explicit human approval gates.

Weekly cadence

Plan on one weekly cycle: define and verify the concept; storyboard; build and preview; record and edit; review and publish. Adjust the cadence to your available teaching time rather than sacrificing correctness.

17. Operating checklists and templates

Episode brief template

Episode ID:

Working title:

Target audience:

Central question:

Prerequisite knowledge:

Learning outcomes:

Common misconception:

Key equations:

Assumptions and scope:

Narrative hook:

Planned scenes:

Practice question:

Language:

Target duration:

Success criteria:

Scene card template

Scene ID and objective:

Starting visual state:

Narration:

Objects and labels:

Equations and units:

Action or transformation:

What the learner should notice:

Duration estimate:

Transition:

Physics risks:

Visual risks:

Acceptance test:

Physics review template

Claim:

Definitions:

Assumptions:

Derivation:

Dimensional check:

Sign/direction check:

Limiting cases:

Numerical or independent check:

Domain of validity:

Potential misconception:

Reviewer and date:

Status: draft / needs revision / approved

Release checklist

- [ ] Physics approved. [ ] Script and storyboard approved. [ ] Preview reviewed at normal speed. [ ] No clipping or unreadable text. [ ] Narration synchronized. [ ] Audio checked. [ ] Captions corrected. [ ] Export checked start-to-finish. [ ] Thumbnail and title reviewed. [ ] Sources and asset licenses recorded. [ ] Source commit/tag created. [ ] Project archived.

Definition of done

A task is not done because AI produced code. It is done when the code runs in the intended environment, the result meets acceptance criteria, scientific and visual checks pass, changes are committed, and the next person can understand how to reproduce it.

18. Official documentation and further study

Primary references

Manim Community documentation: https://docs.manim.community/ . Manim Community site: https://www.manim.community/ . Google Colab: https://colab.research.google.com/ . GitHub: https://github.com/ . GitHub Codespaces: https://github.com/features/codespaces . Python documentation: https://docs.python.org/3/ . SymPy documentation: https://docs.sympy.org/ . NumPy documentation: https://numpy.org/doc/ . FFmpeg documentation: https://ffmpeg.org/documentation.html . DaVinci Resolve product and training: https://www.blackmagicdesign.com/products/davinciresolve . Audacity: https://www.audacityteam.org/ .

How to use references

Documentation and installation instructions change. Use official documentation for current setup commands and API details. Treat AI-generated commands as proposals to verify. Keep a note of the documentation version or access date for instructions that are important to reproduce the build.

Final principle

> The long-term advantage of SaemPhysics will not come from using the largest number of AI tools. It will come from a coherent visual language, verified physics, memorable teaching, reusable engineering, and a production system that lets you spend more of your time on explanation rather than repetitive setup.

One-page operating doctrine

- **Physics first:** No equation or simulation is approved because an AI generated it.
- **One scene, one purpose:** Every animation must help the learner notice or infer something.
- **Small, testable artifacts:** Brief, derivation, scene card, code, audio, export and review record are separate.
- **Preview early:** Render a low-quality scene before investing in a long episode.
- **Human approval gates:** Approve objective, physics, storyboard, preview and final export.
- **Repository is the memory:** Document setup and decisions; do not rely on chat history.
- **Automate after learning:** Automate stable, repetitive steps only after the manual workflow works.
- **Improve from evidence:** Track time, errors, retention and student feedback.

The future roadmap: SaemPhysics becomes an AI-powered mathematical and scientific storytelling studio

Yes. I would take the project far beyond Manim animations and AI-assisted scripts. The long-term vision is to build a knowledge-to-lecture engine that studies trusted educational sources, organizes mathematical and scientific ideas into a structured knowledge base, and turns those ideas into original, visually rich lectures.

Think of it as combining five capabilities:

* The depth of MIT OpenCourseWare and university-level teaching.

* The conceptual questioning and physical intuition associated with Feynman.

* The mathematical visual storytelling associated with 3Blue1Brown.

* The rigorous, structured development of mathematics and theoretical physics found in great lecture courses.

* An engineering system that can generate, test, animate, narrate, and improve lectures.

The important distinction is that the system should not simply summarize lectures or imitate famous teachers. It should learn from their publicly available ideas, pedagogical principles, mathematical methods, and scientific contributions, then create original explanations in the SaemPhysics voice.

1\. The long-term system architecture

Layer 1 — Scientific knowledge acquisition

MIT OCW, open textbooks, lecture transcripts, public-domain works, papers, and trusted references

Layer 2 — Knowledge and reasoning engine

Concept maps, prerequisite graphs, derivations, definitions, proofs, misconceptions, and source citations

Layer 3 — Lecture design engine

Learning objectives, narrative, intuition, proofs, demonstrations, examples, and problem sets

Layer 4 — Mathematical and scientific computing

Python, SymPy, NumPy, numerical solvers, computer algebra, symbolic verification, and simulations

Layer 5 — Visual lecture compiler

Manim, diagrams, plots, 2D/3D geometry, camera choreography, narration, captions, and editing

Layer 6 — Verification and release

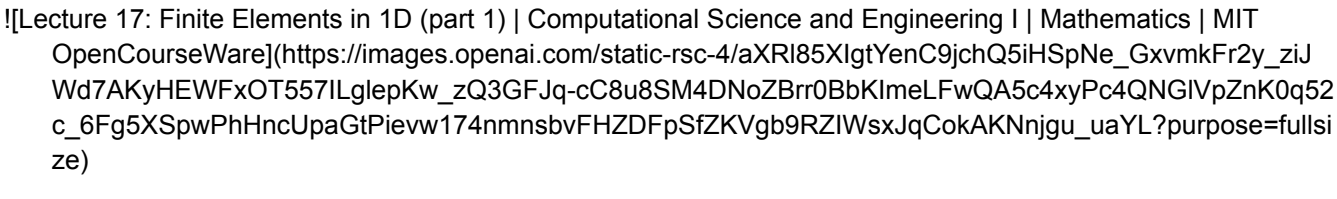
Scientific review, rendering tests, pedagogical evaluation, source attribution, and teacher approval

The most valuable part is not the animation engine alone. It is the connection between verified knowledge, pedagogical reasoning, and a reusable visual representation of mathematics.

2\. The knowledge engine: learning from MIT OpenCourseWare and great scientific works

The first major capability should be an automated open educational resource ingestion system.

MIT OpenCourseWare is an excellent starting point, but it should be one source in a broader scientific library. Different sources serve different purposes: a lecture may provide intuition, a textbook may provide a rigorous proof, and a problem set may reveal what learners are expected to do.

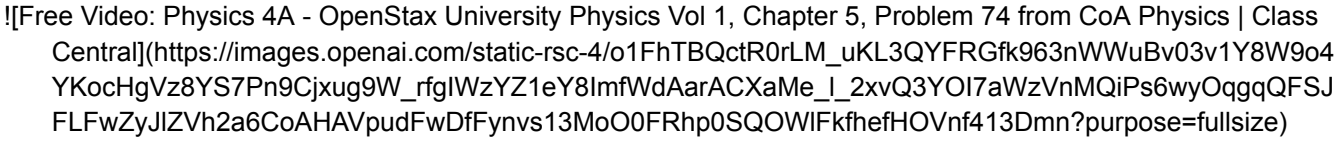


MIT OpenCourseWare

Course structure and depth

Extract permitted lecture notes, course outlines, problem sets, reading lists, and available transcripts. Use these to map prerequisites and build a coherent sequence of topics.

[MIT OCW](https://ocw.mit.edu/?utm_source=chatgpt.com)



Open textbooks

Definitions, explanations, and worked examples

Use openly licensed resources such as OpenStax to cross-check foundational explanations and source material.

[OpenStax](https://openstax.org/?utm_source=chatgpt.com)

![Isaac Newton | The mathematical principles of natural philosophy...,](https://images.openai.com/static-rsc-4/HKrSDZSpdy0x0VkYa58_iJ_nSjHj9FffzE-J0WJ815abVaUUuRTb7kMfr51RCw1iz0gncauESiMQdkxWqZYI_KwhGyPieGduUc2x7K-uaWbOwR6piI3apx0l1i0rqT2sYlDr4QU0fRrBUCCxFAqSYulSl1PC0RLHcZfxt1jXMTzO2-L9WmXUMGQsYV9YlgZp?purpose=fullsize)

Historical scientific works

Original discoveries and mathematical ideas

Study works by Newton, Maxwell, Faraday, Euler, Gauss, Fourier, Noether, Einstein, and other foundational thinkers. Preserve historical context and distinguish original notation from modern formulations.

[Project Gutenberg](https://www.gutenberg.org/?utm_source=chatgpt.com)

![OpenAI взялась за научный сектор. Компания представила Prism — бесплатную рабочую область для учёных, изначально созданную с использованием ИИ](https://images.openai.com/static-rsc-4/HCA_bAUZAeBChNImbntVBGR0NH77p04IpHTTjmj9bPWBMFLxbyN1OOYfFmOEdTtZXOGaW_VwoRATX70TdylJUatSErsjoqE7Hl7R0v1h7s2prq6ODs4mJnMWmTDzuGgDcId9PVF2ka2aGrjfsk-89niw_bZvCGx1cemFHoyPJuM__1qZnxi3zGxFlBUQtQH9?purpose=fullsize)

Research papers and specialist references

Advanced material and modern developments

Use sources such as arXiv when you move into university mathematics, theoretical physics, numerical methods, or computer science. Research claims require careful review.

[arXiv](https://arxiv.org/?utm_source=chatgpt.com)

What should the extractor actually do?

It should not simply download thousands of documents and feed them to a chatbot. It should transform source material into structured, traceable knowledge.

1\. Discover and catalog

Find course pages, metadata, lecture notes, transcripts, and permitted downloads. Record source URLs, authors, dates, licenses, and retrieval details.

2\. Extract and normalize

Process HTML, PDFs, text, captions, equations, diagrams, and notebooks. Keep equations and section headings intact wherever possible.

3\. Identify claims and derivations

Extract definitions, theorem statements, assumptions, equations, proofs, examples, and open questions. Attach source locations to each item.

4\. Build the concept graph

Connect prerequisites, related ideas, alternate derivations, applications, common misconceptions, and follow-on topics.

5\. Verify and approve

Run mathematical checks where possible, compare independent sources, flag disagreements, and require human approval for important results.

The knowledge record

Every concept should eventually have a structured record like this:

```yaml
concept_id: electrostatics.point_charge_field
title: Electric field of a point charge
domain: physics.electromagnetism
level:
  - NEET
  - JEE
```

```
  - undergraduate

prerequisites:
  - vectors
  - coulomb_law
  - inverse_square_law

definitions:
  - electric_field

equations:
  - expression: "E = k*Q/r^2"
    kind: magnitude
    assumptions:
      - point_charge
      - r_greater_than_zero

visualizations:
  - radial_field_vectors
  - inverse_square_graph

misconceptions:
  - confusing_force_and_field
  - adding_vector_magnitudes_directly

sources:
  - source_id: verified_source_record
    section: relevant_section

review_status: needs_teacher_approval
```

This is illustrative YAML, not a complete software specification. The important design principle is that a concept record carries its prerequisites, assumptions, source trail, and verification status along with its equations.

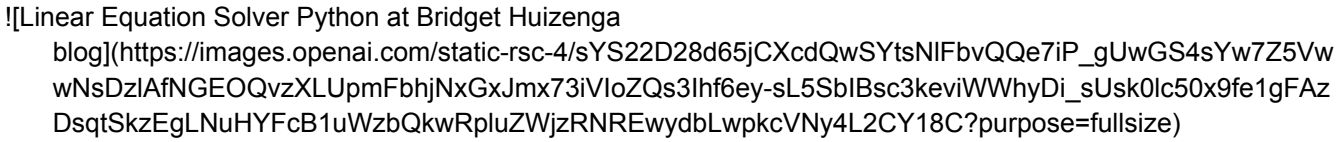
Critical rule: the system must distinguish a source's actual statement from an AI-generated explanation or inference. These should never be silently merged.

Also, MIT OCW material is not automatically unrestricted for every reuse. Follow the specific content license and terms, respect attribution requirements, and check restrictions before downloading or redistributing content. The engine should generate original lessons from verified understanding rather than reproduce source lectures.

3\. The mathematical reasoning engine

This is the capability that will distinguish your studio from an ordinary AI video generator.

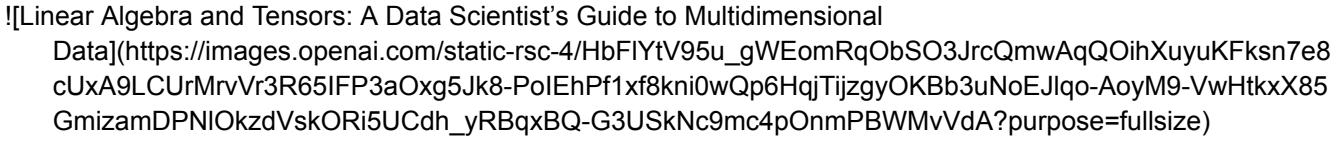
A general-purpose language model can explain an equation fluently while making a subtle algebraic or physical error. Therefore, your studio should use a collection of specialized mathematical tools rather than rely on language generation alone.



Symbolic mathematics

Use SymPy for symbolic algebra, differentiation, integration, equation solving, and simplification.

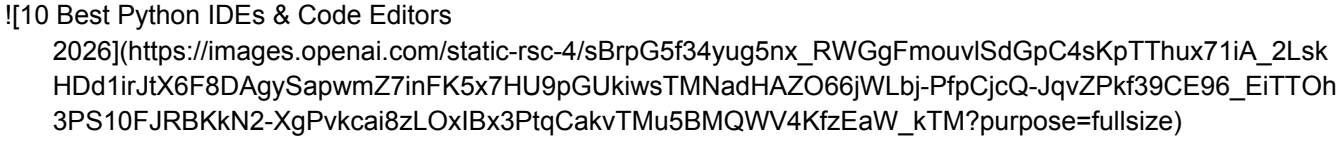
Example: independently differentiate a proposed position function to check its velocity and acceleration.



Numerical computation

Use NumPy and numerical solvers to explore physical systems and compare approximations with known solutions.

Example: compare an approximate pendulum solution with a numerical integration of the nonlinear equation.



Computational experiments

Use notebooks for plots, parameter sweeps, counterexamples, and numerical tests.

Example: explore how a graph changes when initial conditions or physical parameters change.

Formal verification — later phase

Explore Lean for formal proofs in mathematics when the knowledge library becomes mature.

This is a long-term capability, not a prerequisite for making your first physics videos.

The reasoning engine should run several different checks:

* Symbolic: does the algebraic manipulation preserve the expression under the stated conditions?

* Dimensional: do the units match?

* Numerical: does the result agree with independently computed test cases?

* Limiting case: does the result behave sensibly at relevant extremes?

* Physical: do the assumptions, directions, boundary conditions, and domain make sense?

* Pedagogical: can the intended learner follow the reasoning?

No single test proves a physical claim. Together, they provide stronger safeguards.

4\. The lecture compiler: from understanding to video

This should become the heart of your future platform.

Instead of prompting an AI with “make a video about calculus,” you give it a verified concept and a learner profile. The system plans a lesson, generates a storyboard, builds animations, and checks whether the resulting lecture meets its learning objectives.

I would call this component the SaemPhysics Lecture Compiler.

Its job is to compile structured educational knowledge into a sequence of visual teaching actions.

A. The compiler's intermediate representation

Compilers use intermediate representations because they separate the meaning of a program from the machine that executes it. Your lecture system can use the same idea.

For example, the idea “the derivative is the instantaneous rate of change” should not be stored only as a paragraph of prose. It should be represented as a concept with a mathematical definition, a visual explanation, prerequisites, and an assessment.

A lecture representation might look like this:

```yaml
lecture:
  topic: derivative_as_rate_of_change
  audience: JEE_Advanced
  objective: >
    Explain why the derivative represents an
    instantaneous rate of change.

  beats:
    - id: changing_quantity
      teaching_goal: build_intuition
      visual: moving_point_on_curve
      question: "How fast is the quantity changing here?"

    - id: secant_to_tangent
      teaching_goal: introduce_limit
      visual: shrinking_secant_interval
      equation: "f'(x) = lim[h->0] (f(x+h)-f(x))/h"

    - id: physical_interpretation
      teaching_goal: connect_to_motion
      visual: position_and_velocity
      equation: "v(t) = dx/dt"
```

```
  verification:
    require_symbolic_check: true
    require_teacher_approval: true
```

This is a conceptual specification. A production system would validate the schema, resolve each visual into a supported animation component, and reject incomplete or unverified content.

B. How the compiler should operate

01

Select the concept

Retrieve the relevant definitions, prerequisite concepts, equations and source passages.

02

Plan the explanation

Choose the question, intuition, formal development, examples and desired learning outcomes.

03

Choose visual representations

Map concepts to graphs, vectors, geometry, particles, coordinate systems, diagrams or 3D objects.

04

Generate scene specifications

Create explicit scene IDs, equations, labels, narration cues, timing targets and transition instructions.

05

Generate and execute code

Produce Manim or simulation code from approved components; render a low-quality preview.

06

Evaluate and revise

Run mathematical checks, inspect visual output, revise unclear sections and obtain teacher approval.

07

Package the lecture

Combine animation, narration, captions, exercises, references and publication metadata.

The compiler should generate a reviewable intermediate artifact at every stage, rather than jumping straight from a topic prompt to a final MP4.

That gives you the ability to replace one component without rebuilding the entire system. You could improve the narration model while keeping the storyboard, or change the rendering engine while preserving the verified lecture specification.

5\. The great thinkers library

You mentioned wanting to build on the understanding of great mathematicians and physicists throughout history. I would make this a separate layer called the History and Methods of Ideas Library.

The objective is not to create artificial replicas of famous people. It is to study the different intellectual approaches that made their work powerful and turn those approaches into useful teaching methods.

Richard Feynman — physical intuition

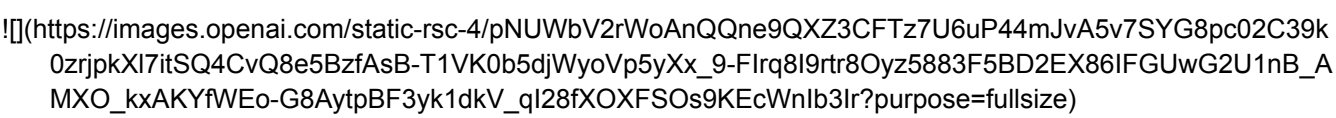
* Start with a concrete physical question.

* Challenge assumptions and vague explanations.

* Connect mathematical formalism to observable consequences.

* Ask what would happen if a parameter changed.

Studio feature: a concept-first lesson planner.



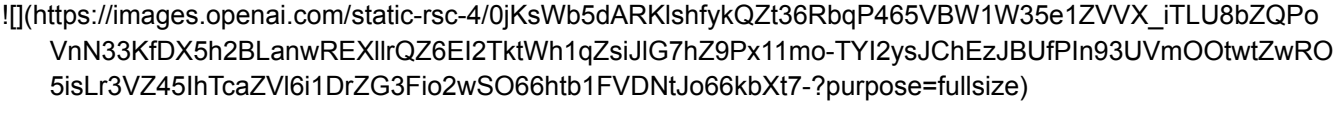
Leonhard Euler — mathematical structure

* Develop relationships systematically.

* Connect algebra, geometry, functions and mechanics.

* Show how the same mathematical structure appears in different problems.

Studio feature: a cross-domain concept connector.



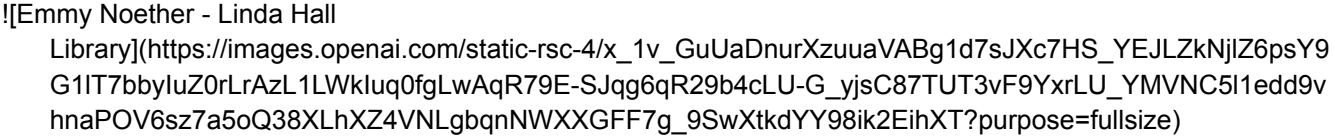
Michael Faraday — experimental intuition

* Begin with phenomena and experimental questions.

* Build visual models of fields and interactions.

* Explain the role of evidence in developing physical ideas.

Studio feature: experiment-to-model lesson templates.

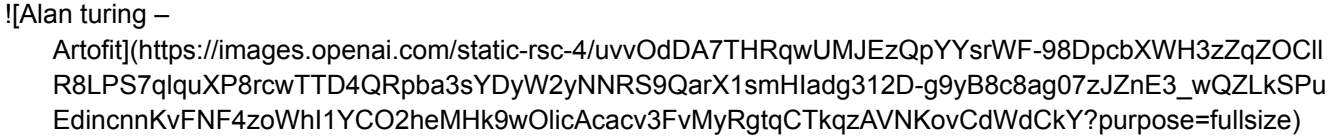


Emmy Noether — symmetry and conservation

* Reveal the structural connection between symmetry and conserved quantities.

* Connect transformations to deeper mathematical principles.

Studio feature: symmetry visualization and advanced conceptual pathways.



Alan Turing — computation and algorithms

* Turn abstract procedures into explicit steps.

* Explore what can be computed, how algorithms operate, and where limitations arise.

Studio feature: algorithm animations and computational thinking lessons.

You can extend this library with Newton, Maxwell, Gauss, Fourier, Hamilton, Lagrange, Einstein, Dirac, Hilbert, Ramanujan, Turing, Shannon, and many others.

Each thinker record should contain documented ideas, historical context, representative works, source references, and a set of teaching principles derived from the material. Do not attribute invented quotations, methods, or opinions to them.

For visual design, learn from broad principles such as smooth transformations, geometric clarity, carefully timed equation reveals, and explanatory motion. Create your own visual language instead of copying another creator's distinctive scenes or presentation.

6\. The roadmap: 0–24 months

I would build this in stages, with each phase producing a working capability. The time estimates below are planning targets for a solo teacher using AI coding assistance, not guarantees.

Phase 1

Months 0–2

The animation foundation

First priority

Build the GitHub repository, Manim environment, reusable components, render presets, and physics review templates. Produce 3–5 short animations.

Exit condition: a fresh project can reliably render a verified scene from reusable components.

Phase 2

Months 2–4

The mathematics and simulation engine

Add SymPy checks, numerical experiments, graphing utilities, model tests, and a structured scene specification. Begin building a reusable library of physics models.

Exit condition: a selected concept can be tested computationally and visualized with repeatable results.

Phase 3

Months 3–6

The educational knowledge base

Build a source catalog and an ingestion pipeline for a small set of permitted MIT OCW courses and open textbooks. Extract concepts, equations, prerequisites, source passages, and learning objectives.

Exit condition: you can retrieve evidence for a topic and trace important claims back to their source.

Phase 4

Months 5–9

The lecture compiler MVP

Given a topic, audience, duration and learning objective, generate a sourced lecture outline, storyboard, scene specification and initial Manim code.

Exit condition: one complete lecture passes the scientific and pedagogical review process without requiring you to write every scene from scratch.

Phase 5

Months 9–15

Multimodal scientific production

Integrate narration timing, caption correction, diagram and image ingestion, 3D visualization where justified, and automated render testing. Build a searchable library of approved scenes.

Exit condition: a complete episode can be assembled through a documented pipeline with controlled human review.

Phase 6

Months 15–24

The cross-disciplinary scientific studio

Extend the system to calculus, linear algebra, probability, algorithms, data structures, computer architecture, numerical methods, classical mechanics, electromagnetism, and introductory quantum mechanics.

Exit condition: the same architecture supports several disciplines using domain-specific models, tests, visual components and review rules.

Do not wait until Phase 6 to publish. Publish as soon as Phase 1 produces a good video. Real student questions and audience feedback should influence every subsequent phase.

7\. The technical stack I would choose

Avoid unnecessary complexity at the beginning. The architecture should be modular enough to grow, but simple enough for you to operate as a physics teacher.

Capability	Initial choice	Later upgrade

AI reasoning	Gemini + DeepSeek	Model routing and specialized review agents
Source ingestion	Python + permitted downloads	Scheduled extraction and OCR pipelines
Knowledge storage	Markdown, YAML, SQLite	PostgreSQL + semantic retrieval + concept graph
Mathematics	SymPy + NumPy	Specialized solvers and formal verification
Animation	Manim Community	Reusable 2D/3D renderers
Development	GitHub + Codespaces	Automated testing and render workers
Prototyping	Google Colab	Reproducible cloud jobs
Lecture generation	Templates + structured scene specifications	Lecture compiler and visual planning agents
Publishing	Manual editing and upload	Controlled release packaging and analytics

You do not need a sophisticated vector database on day one. Start with structured files and searchable source records. Add semantic retrieval when ordinary search is no longer sufficient.

8\. Your first 30 days

The biggest danger is designing a grand system that never produces a finished video. Your first month should prove the entire process on a small scale.

Implementation checklist

0/12 complete

Week 1

Create the engineering foundation

Set up GitHub and the development environment

Render a Manim equation, vector and graph

Create the project and episode templates

Week 2

Build your visual components

Create reusable equation and vector animations

Implement a simple numerical physics model

Add basic tests and render instructions

Week 3

Build your first knowledge-to-video workflow

Choose one topic and collect permitted source materials

Write and verify the derivation

Generate a storyboard and render the scenes

Week 4

Publish and evaluate

Record narration and edit the episode

Correct captions and check the final export

Document errors, time spent and improvements

9\. Download the future roadmap

I have also prepared a separate roadmap document to accompany your original architecture textbook. It goes into the knowledge ingestion pipeline, lecture compiler, thinker library, data architecture, evaluation safeguards, and staged implementation plan.

Future Roadmap: Knowledge-to-Lecture Engine

PDF edition for reading and reference

Open the future roadmap (PDF)

Editable roadmap

Word edition for expanding into a living engineering specification

Open the editable roadmap (DOCX)

10\. The most important strategic decision

Build the system in this order:

* A reliable animation engine.

* A tested mathematical and physical model library.

* A source-grounded scientific knowledge base.

* A lecture compiler that turns verified knowledge into storyboards and scenes.

* A production pipeline that assembles and checks the final video.

* Cross-disciplinary expansion into mathematics, physics and computer science.

Do not begin by trying to make AI autonomously understand all of MIT OCW. Prove that it can accurately ingest, explain, verify and animate one well-defined concept. Then expand that capability systematically.

Your real long-term asset will be a growing library of verified concepts, reusable mathematical models, visual components, teaching patterns and tested lecture specifications. Each new video will improve the studio, making the next one easier to create.

That is how SaemPhysics can grow from a YouTube teaching channel into a serious, scalable scientific visual education platform.